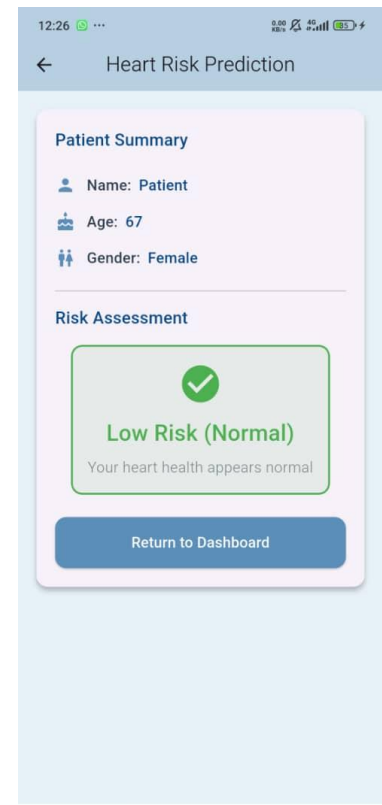
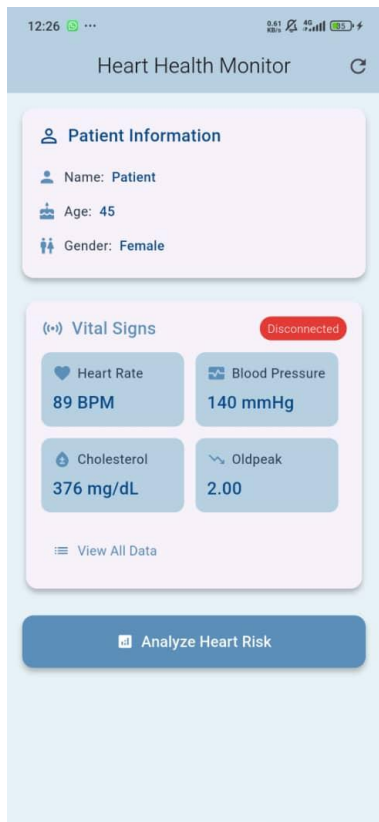


What I did?

- get the **Heart Disease model** from : <https://www.kaggle.com/code/zahidmughal2343/heart-disease-prediction-using-random-forest/input>

- 1- Get .csv file
- 2- train\_model.py and get heart\_model.h5
- 3- TFLiteConverter to get heart\_model.tflite
- 4- Add heart\_model.tflite to flutter app in assets
- 5- pubspec.yaml
- 6 – I did mock sender to send fake data through mqtt (broker.hivemq.com) to the application then the ML will work inside the app to predict



Xray model: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia?resource=download>

- 1- download as zip code (2GB)
- 2- I UPLOADED to my drive and connect with Google Colab

from google.colab import drive

drive.mount('/content/drive') => /content/drive/My Drive/archive.zip

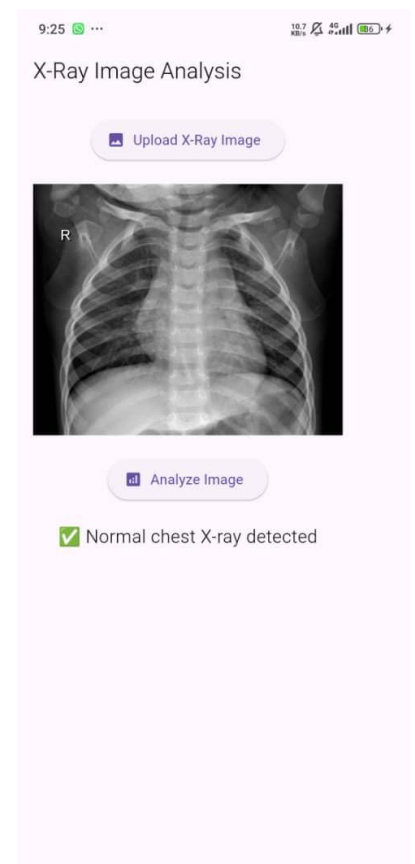
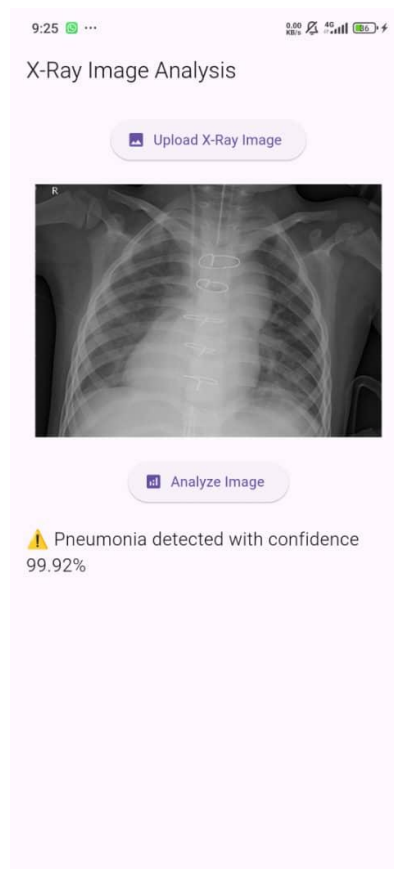
- 3- Unzip the file
- 4- Be sure that files inside
- 5- Convert the path /content/chest\_xray\_clean/
- 6- Libraries,paths
- 7- ImageDataGenerato and resize the images
- 8- CNN Model
- 9- Train : history = model.fit(  
train\_generator,  
epochs=10,  
validation\_data=val\_generator)

92.6 %

## 10 – FliteConverter

I did an app through I can upload an X-Ray image and a ML model will run inside the app to predict if there is a Pneumonia or it's normal

I did this just because I want to Work on [AWS Rekognition](#)



The sensors that we need to our project:

| Sensor Name           | Purpose / What it Measures                      | For Parameter              | Example Model(s)                                  |
|-----------------------|-------------------------------------------------|----------------------------|---------------------------------------------------|
| Heart Rate Sensor     | Measures heart beats per minute                 | thalach                    | Pulse Sensor,<br>MAX30100,<br>MAX30102            |
| Blood Pressure Sensor | Measures systolic and diastolic blood pressure  | trestbps                   | Digital BP monitors<br>(Bluetooth/USB interfaces) |
| ECG Sensor            | Measures electrical activity of the heart (ECG) | restecg,<br>oldpeak, slope | AD8232                                            |

My idea is to search about them in the lab

If we couldn't find them we can put the values random

**manually input:**

- age
- sex
- cp (chest pain type)
- exang (exercise-induced angina)

*Also these parameters because we Cannot be measured without laboratory blood tests:*

- **chol** (serum cholesterol)
- **fbs** (fasting blood sugar)
- **ca** (number of major vessels colored by fluoroscopy) *medical imaging (fluoroscopy)*
- **thal** (thalassemia type)

**From sensors :**

- trestbps (resting blood pressure) — from Blood Pressure Sensor
- restecg (resting ECG results) — from ECG device
- thalach (max heart rate achieved) — from Heart Rate Sensor during exercise test
- oldpeak (ST depression induced by exercise) — from ECG device
- slope (slope of ST segment) — from ECG device

**LAST UPDATE: SEND BY WHATSAPP**

## AWS Amplify Auth using Cognito in Flutter

I integrated AWS Amplify Auth (Cognito) into our Heart Health Monitor project to handle secure user authentication both offline and online.

### ❖ Why Amplify Auth?

- Users can use AWS Cognito to sign in online using Amplify.
- User credentials (e.g., `userId` or tokens) are stored locally on successful sign-in with **SharedPreferences** or **SecureStorage**.
- If **no internet** => the app operates using stored credentials to authenticate the user and execute the app in Edge mode.
- If **internet connection becomes available again** => it checks the token and syncs with AWS automatically.

---

### Login Flow:

#### ❖ First-time Login (with Internet) :

1. User enters password and email.
2. App calls `Amplify.Auth.signIn(.,.)`.
3. If successful:
  - o User data is saved locally.
  - o The app keeps executing (Edge or Cloud mode depending on connectivity).

#### ❖ Subsequent Login (without Internet):

1. App checks local storage.
  2. If a previous user is found → grant access and trigger Edge mode.
  3. When internet returns → re-sync with AWS.
-

## Setting up Amplify in Flutter

### 1. Install Amplify CLI

```
npm install -g @aws-amplify/cli
```

### 2. Initialize Amplify in Flutter project

From the root of your Flutter project, run:

```
amplify init
```

- Choose Yes to continue using Gen 1 (Gen 2 is not currently supported by Flutter).
- Project name: flutterhealthapp
- Language: Flutter
- Environment: dev (or any name)
- Choose AWS profile or login when prompted.

### 3. Configure Authentication

```
amplify add auth
```

- Choose: Default configuration
- Then run:

```
amplify push
```

---

## AWS Access Credentials Setup

When prompted for accessKeyId and secretAccessKey:

### 1. Go to IAM Console

**2.Create a new one**

**3.Add AdministratorAccess to this user**

**4.Under Security Credentials, generate a new Access Key**

**5.Copy:**

O Access Key ID

O Secret Access Key

**6.Back in terminal:**

Setup new user: Yes

accessKeyId: [your key]

secretAccessKey: [your secret]

**•For profile name, I used this name:**

amplify-flutter

Use the AWS profile you wish to use?

I picked amplify-flutter.

---

Flutter Packages: in **pubspec.yaml**: (I edit them a lot )

In your pubspec.yaml:

dependencies:

amplify\_flutter: ^1.3.0

amplify\_auth\_cognito: ^1.3.0

Or add with:

flutter pub add amplify\_flutter amplify\_auth\_cognito

---

### **Add the Amplify Configuration File (Important)**

Once you've executed amplify init and amplify push, there's a file called amplifyconfiguration.dart that gets generated.

You can find it at:

`flutter_heart_app_new/amplifyconfiguration.dart`